Laboratory 10

Tickless Idle & Power Measurement

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (`arm-none-eabi-gcc`) + Make
Estimated time	3–4 hours
CLO mapping	CLO 4 & CLO 5

1. Objectives

By completing this lab you will be able to:

1. **Measure** average current consumption with and without FreeRTOS tickless idle.
2. **Configure** the built-in tickless idle (`configUSE_TICKLESS_IDLE = 1`).
3. **Implement** a custom `vPortSuppressTicksAndSleep` using the MCXN236 LPTMR.
4. **Verify** tick correction accuracy using a 1-second LED toggle and DWT timing.
5. **Estimate** battery life improvement from tickless idle over a 24-hour scenario.

2. Prerequisites

- ☐ Lab 06 complete (DWT cycle counter working).
- ☐ A current probe or NXP MCU-Link power measurement capability **OR** an ammeter in series with the 3.3 V supply rail.
- ☐ ARM GNU Toolchain and `pyocd` working.

> **Note**
>
> **Power measurement alternatives:** If no current probe is available, you can use an MCU-Link Pro board's power measurement feature via MCUXpresso IDE, or a USB power meter on the host port. The relative comparison (with vs. without tickless) is more important than the absolute value.

3. Background

3.1. The Tick Interrupt Problem

FreeRTOS generates a SysTick interrupt every 1 ms (at 1 kHz tick rate). Even when all tasks are blocked, the CPU wakes every 1 ms to run the tick handler — about 100–200 cycles of overhead, repeated 1 000 times per second.

On a system where the active task runs only 2 ms every 500 ms (0.4% duty cycle), the tick alone dominates power consumption in the idle period.

3.2. Tickless Idle Flow

```
Scheduler finds all tasks blocked:
  1. Compute xExpectedIdleTime = ticks until next task unblocks
  2. Call vPortSuppressTicksAndSleep(xExpectedIdleTime)
     a. Disable SysTick
     b. Program LPTMR for xExpectedIdleTime (in 32 kHz ticks)
     c. __WFI with SLEEPDEEP
     d. Wake (LPTMR or other interrupt)
     e. Read elapsed LPTMR ticks
     f. vTaskStepTick(elapsed) -- correct the tick count
```

```
     g. Re-enable SysTick
  3. Resume normal scheduling
```

4. Project Structure

```
lab10_tickless/
  Makefile
  linker_MCXN236.ld
  src/
    main.c                    <- task creation, LED init, DWT init
    adc_task.c/.h             <- ADC sample task (500 ms period)
    led_toggle_task.c/.h      <- 1-second LED (timing accuracy test)
    tickless_port.c/.h        <- Part C: custom vPortSuppressTicksAndSleep
    FreeRTOSConfig.h
    uart.h / uart.c
  FreeRTOS-Kernel/
```

5. Part A — Baseline Current Measurement (No Tickless)

5.1. Step 1 — Configure with Tickless Disabled

In `FreeRTOSConfig.h`:

```
#define configUSE_TICKLESS_IDLE          0
#define configTICK_RATE_HZ               1000
```

5.2. Step 2 — Implement the ADC Task

```
void vADCTask(void *pv)
{
    TickType_t xLastWake = xTaskGetTickCount();
    uint32_t sample = 0;
    for (;;) {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(500));
        /* Simulate ADC read: ~50 cycles */
        sample = (sample + 37) % 4096;
        uart_printf("[ADC] sample=%lu\r\n", sample);
    }
}
```

This task is active for ∼50 cycles (0.33 µs) every 500 ms — duty cycle ≈ 0.000067%.

5.3. Step 3 — Measure Current

Connect your ammeter or current probe in series with the board's 3.3 V supply. Run for 30 seconds and record minimum and average current.

Checkpoint A

Record average mA with `configUSE_TICKLESS_IDLE = 0`.

Question A1

The tick fires 1 000 times per second. Each tick handler takes roughly 150 cycles. What fraction of total CPU time is consumed by tick handling alone? Convert to a percentage.

6. Part B — Built-In Tickless Idle

6.1. Step 1 — Enable Built-In Tickless

In `FreeRTOSConfig.h`:

```
#define configUSE_TICKLESS_IDLE                  1
```

```
#define configEXPECTED_IDLE_TIME_BEFORE_SLEEP     2
```

No other code changes are needed — the Cortex-M33 FreeRTOS port implements `vPortSuppressTicksAndSleep` using SysTick reload.

6.2. Step 2 — Measure Current Again

Repeat the same 30-second measurement.

Checkpoint B

Current measurement with `configUSE_TICKLESS_IDLE = 1`.

Question B1

How much did the average current drop? Express as a percentage reduction.

Question B2

With tickless enabled, the CPU still wakes every 500 ms for the ADC task. Calculate the total number of CPU wakeups per hour:

- Without tickless: _____ (tick + task)
- With tickless: _____ (task only)

7. Part C — Custom LPTMR Tickless Implementation

The built-in port uses SysTick (which stops in Deep Sleep on some MCU configurations). A better approach is to use the LPTMR, which runs from the 32.768 kHz VBAT clock and survives Deep Sleep.

7.1. Step 1 — Switch to Custom Tickless

In `FreeRTOSConfig.h`:

```
#define configUSE_TICKLESS_IDLE          2   /* custom implementation
    */
```

7.2. Step 2 — Implement `vPortSuppressTicksAndSleep`

In `tickless_port.c`, implement all six steps:

```
#include "FreeRTOS.h"
#include "task.h"
#include "fsl_lptmr.h"

#define LPTMR_CLOCK_HZ   32768UL   /* VBAT 32 kHz oscillator */

void vPortSuppressTicksAndSleep(TickType_t xExpectedIdleTime)
{
    /* --- 1. Disable SysTick --- */
    SysTick->CTRL &= ~SysTick_CTRL_ENABLE_Msk;

    /* --- 2. Program LPTMR --- */
    uint32_t lptmr_ticks = (uint32_t)(
        ((uint64_t)xExpectedIdleTime * LPTMR_CLOCK_HZ)
        / (uint64_t)configTICK_RATE_HZ);

    if (lptmr_ticks == 0) {
        SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;
        return;
```

```
    }

    LPTMR0->CSR = 0;
    LPTMR0->CMR = lptmr_ticks - 1;
    LPTMR0->CSR = LPTMR_CSR_TEN_MASK | LPTMR_CSR_TIE_MASK;

    /* --- 3. Enter Deep Sleep --- */
    SCB->SCR |= SCB_SCR_SLEEPDEEP_Msk;
    __DSB();
    __WFI();
    SCB->SCR &= ~SCB_SCR_SLEEPDEEP_Msk;

    /* --- 4. Stop LPTMR and read elapsed time --- */
    LPTMR0->CSR &= ~LPTMR_CSR_TEN_MASK;
    uint32_t elapsed_lptmr = LPTMR0->CNR;
    LPTMR0->CSR |= LPTMR_CSR_TCF_MASK;

    /* --- 5. Correct FreeRTOS tick count --- */
    TickType_t elapsed_ticks = (TickType_t)(
        ((uint64_t)elapsed_lptmr * configTICK_RATE_HZ)
        / (uint64_t)LPTMR_CLOCK_HZ);

    if (elapsed_ticks > 0)
        vTaskStepTick(elapsed_ticks);

    /* --- 6. Re-enable SysTick --- */
    SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;
}

void LPTMR0_IRQHandler(void)
{
    LPTMR0->CSR |= LPTMR_CSR_TCF_MASK;   /* clear flag -- wake only
       */
}
```

7.3. Step 3 — Verify Timing Accuracy with LED Toggle

```
void vLEDTask(void *pv)
{
    TickType_t xLastWake = xTaskGetTickCount();
    uint32_t toggle_count = 0;
    DWT_Enable();
    uint32_t prev_cycles = DWT->CYCCNT;

    for (;;) {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(1000));
        uint32_t now_cycles = DWT->CYCCNT;
        uint32_t elapsed_ms = (now_cycles - prev_cycles) / 150000;
        prev_cycles = now_cycles;

        GPIO_TogglePinsOutput(BOARD_LED_GPIO, 1u << BOARD_LED_PIN);
        toggle_count++;
        uart_printf("[LED] toggle #%lu  actual period = %lu ms\r\n",
                    toggle_count, elapsed_ms);
    }
}
```

Run for 60 seconds and observe whether the LED period drifts from 1 000 ms.

Checkpoint C

UART from the LED task showing 60 toggle events with period values (must stay within 1 000 ± 2 ms).

? Question C1

The LPTMR runs at 32.768 kHz. The tick is 1 kHz. What is the maximum rounding error when converting between LPTMR ticks and FreeRTOS ticks? Express as ± ms.

? Question C2

`vTaskStepTick` must be called with a value **less than** `xExpectedIdleTime`. Why? What happens if you pass a value that is too large? (Hint: look at `task.c` — `vTaskStepTick` asserts.)

8. Part D — Energy Analysis

8.1. Step 1 — Measure Current with Custom LPTMR Implementation

Repeat the 30-second current measurement from Parts A and B.

8.2. Step 2 — Complete the Comparison Table

Configuration	Avg. current (mA)	Wakeups/hour	Energy (mWh/24h)
No tickless (Part A)		3 600 000	
Built-in tickless (Part B)		7 200	
Custom LPTMR (Part D)		7 200	

Energy (mWh) = average_current (mA) × 3.3 V × 24 h ÷ 1000.

? Question D1

Using your measured currents, how many days would a CR2032 coin cell (225 mAh capacity) power the board under each configuration?

? Question D2

`vTaskDelayUntil` in `vADCTask` wakes the task at exactly 500 ms boundaries. If the LPTMR rounding error from Question C1 is ±0.5 ms per sleep cycle, what is the maximum accumulated timing error after 1 hour? Is this acceptable for a real-time sensor system?

9. Deliverables

#	Item	Details
1	Part A current	Value and methodology description
2	Part B current	Value (tickless = 1)
3	Part C UART	60 LED toggle events, period within 1 000 ± 2 ms
4	Part D table	Completed comparison with measured values
5	`tickless_port.c`	Source listing
6	Written answers	Questions A1, B1–B2, C1–C2, D1–D2

10. Key Concepts Demonstrated

Concept	Where you saw it
Tick interrupt power overhead	Part A current measurement
Built-in SysTick tickless idle	Part B
LPTMR-based custom tickless port	Part C
`vTaskStepTick` tick correction	Part C
Timing accuracy with LPTMR rounding	Part C LED toggle
Energy analysis and battery life estimation	Part D

11. Reference

Resource	Relevant sections
FreeRTOS docs	`configUSE_TICKLESS_IDLE`, `vTaskStepTick`, `vPortSuppressTicksAndSleep`
NXP MCXN236 Reference Manual	Chapter PMC, Chapter LPTMR
ARM Cortex-M33 TRM	§B2.5 Power management, WFI instruction
NXP MCUXpresso SDK	`fsl_lptmr.h`